Module 4: Arrays and Indexing Types (Week 2)

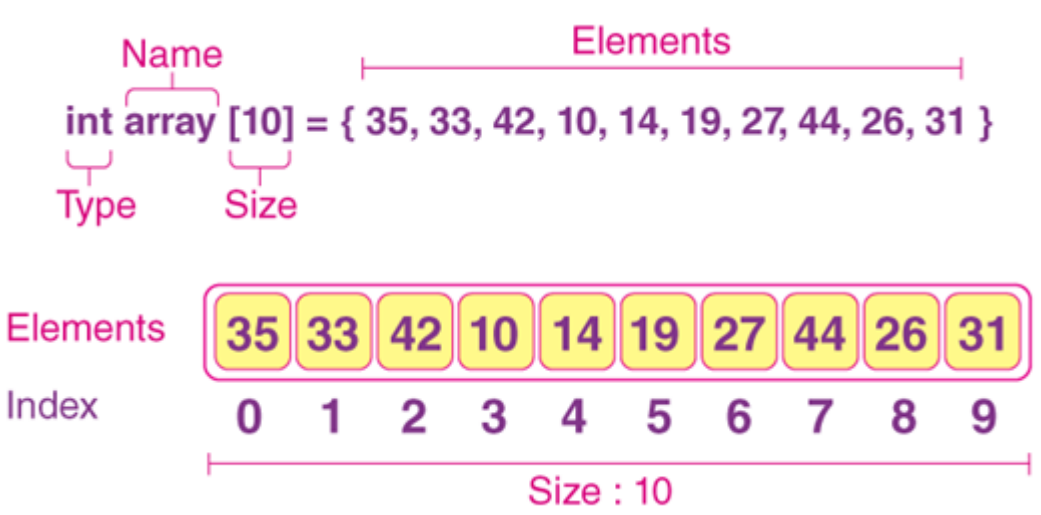
4.1 Introduction

An Array is a linear data structure that collects elements of the same data type and stores them in contiguous and adjacent memory locations. A Linear array is a list of a finite number *n* of homogeneous data elements (i.e., data elements of the same data type) such that

(a) The elements of the array are referenced respectively by an index set consisting of *n* consecutive numbers.

(b) The elements of the array are stored respectively in successive memory locations.

Thus, an Array is a collection of variables of the same data types that are referred to using a common name. An array has a name, size, index and elements.



4.2.1 Properties of an Array

Some of the properties of an array that are:

(i) Each element in an array is of the same data type and carries the same size.

(ii) Elements in the array are stored at contiguous memory locations from which the first element is stored at the smallest memory location.

(iii) Elements of the array can be randomly accessed since we can calculate the address of each element of the array with the given base address and the size of the data element.

4.2.2 The Need for Arrays

Arrays are useful because:

(a) Sorting and searching a value in an array is easier.

(b) Arrays are best to process multiple values quickly and easily.

(c) Arrays are good for storing multiple values in a single variable. In computer programming, most cases require storing a large amount of data of a similar type. To store such an amount of data, we need to define a large number of variables. It would be very difficult to remember the names of all the variables while writing the programs. Instead of naming all the variables with a different name, it is better to define an array and store all the elements into it.

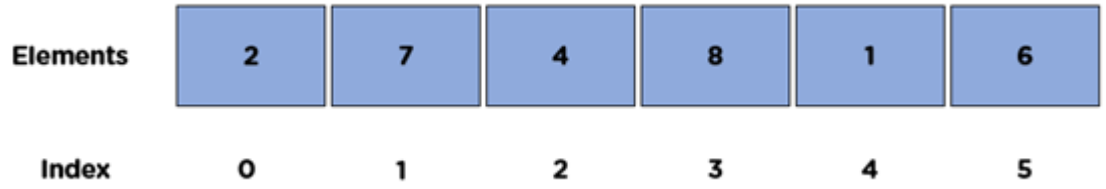
(d) Arrays provide **O(1)** random access lookup time. That means, accessing the 1st index of the array and the 1000th index of the array will both take the same time. This is due to the fact that array comes with a pointer and an offset value. The pointer points to the right location of the memory and the offset value shows how far to look in the said memory.

4.3 Indexing Types and Memory Representation

Two important terms needed to understand the concept of an Array are the array element and array index.

(a) **Element:** Each item stored in an array is called an element. *Elements of an array may be denoted* denoted by subscript notation a_1, a_2, a_3 . . . a_n *by via:*

- Subscription $A_1 \ldots A_n$
- Parenthesis A(1)…A(n)
- Bracket Notation A [1]…A[n]



(b) **Index:** Each location of an element in an array has a numerical index, which is used to identify the element.

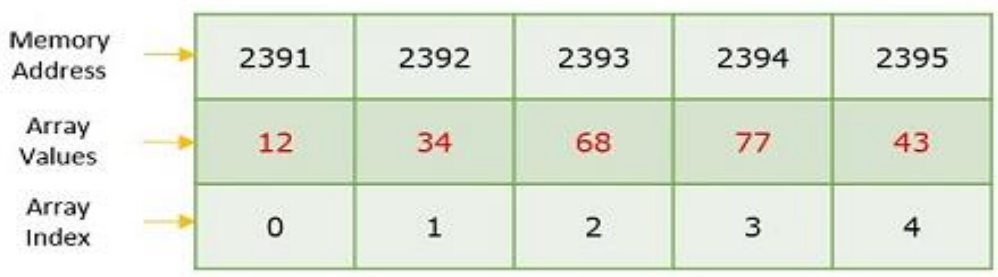


Figure 4.1: Array Elements Array Values) and Array Index

The difference between an *array index* and a *memory address* is that the array index acts like a key value to label the elements in the array. However, a memory address is the starting address of free memory available.

When representing an Array in memory, all the data elements of an array are stored at contiguous locations in the main memory. The name of the array represents the base address or the address of the first element in the main memory. Each element of the array is represented by proper indexing.

We can define the indexing of an array in the below ways:

- 0 (zero-based indexing): The first element of the array will be arr[0].
- 1 (one-based indexing): The first element of the array will be arr[1].
- n (n - based indexing): The first element of the array can reside at any random index number. Usually programming languages allowing *n-based indexing* also allow negative index values and other scalar data types like enumerations, or characters may be used as an array index.

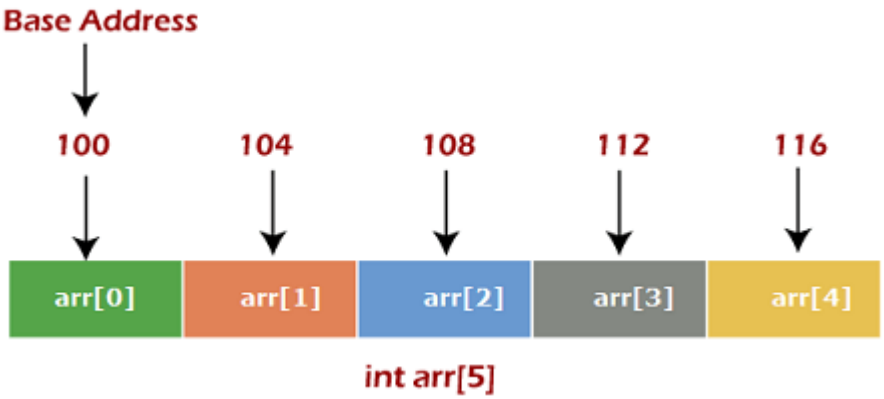


Figure 4.2: Memory Representation of an Array

The Figure 4.2 above shown the memory allocation of an array **"arr"** of size 5. The array follows a 0-based indexing approach. The base address of the array is 100 bytes. It is the address of arr[0]. Here, the size of the data type used is 4 bytes; therefore, each element will take 4 bytes in the memory.

Let ***Arr*** be a linear array in the memory of the computer. The following information given below to access any random element from the array:

- Base Address of the array.
- Size of an element in bytes.
- Type of indexing,

Recall that the memory of computer is simply a sequence of addressed locations.

LOC (Arr[k]) = address of element Arr[k] of the array Arr.

Since the elements of *Arr* are stored in the successive memory cells. Accordingly, the computer does not need to keep track of the address of every element of ***Arr*** array but needs to keep track only of the address of the first element of ***Arr*** array denoted by Base (*Arr*) of array, the base address of ***Arr*** *array*. Using base address, the computer calculates the address of any element of ***Arr*** array by the following formula:

*Byte address of element Arr[i] = base address(Arr) + size * (i - first index/lower bound)*

Here, size represents the memory taken by the primitive data types. As an instance, **int** takes 2 bytes, **float** takes 4 bytes of memory space in C++.

For example, Suppose an array, Arr[-10 . . .+2] having Base address (BA) = 999 and size of an element = 2 bytes, find the location of A[-1].

Loc(Arr[-1]) = 999 + 2 x [(-1) - (-10)]
= 999 + 18
= 1017

4.4 Length vs Size of an Array

The number ***n*** of elements is called length or size of array. If not explicitly stated, we will assume the index set consists of integers 1, 2, 3 …n. In general, the length or the number of data elements of the array can be obtained from the index set by the formula:

$$Length = UB - LB + 1$$

Where *UB* is the largest index, called the *upper bound*, and *LB* is the smallest index, called the lower bound. Note that length = UB when LB = 1.

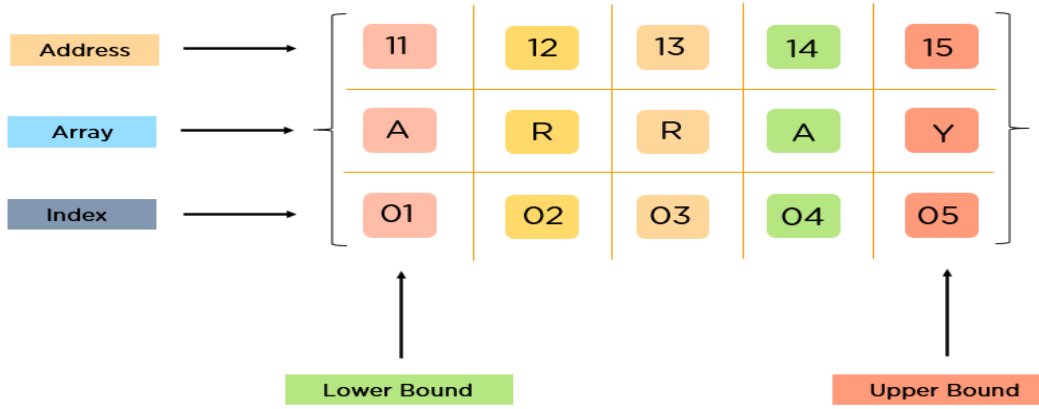


Figure 4.3: Upper and Lower Bound of an Array

4.5 Type of Arrays

Array can be classified based on its dimensions and as static and dynamic array.

4.5.1 Rank/Dimension of an Array (One, Two and Multidimensional Array)

There can be various types of arrays based on their dimensions such as One dimensional, two dimensional and Multidimensional.

(a) One-dimensional Array

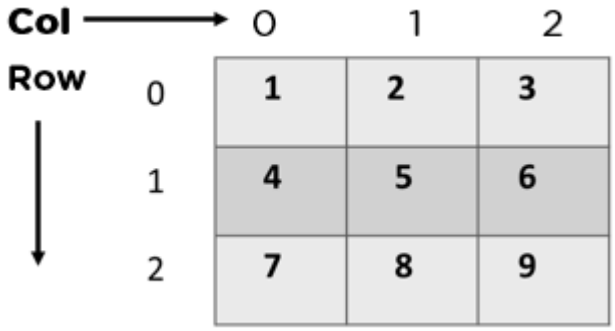
When elements of an array are arranged in a linear fashion, it is called a one-dimensional array. It is the basic array we generally use in many problems.

The syntax of declaring one-dimensional array a one-dimensional array is given as follows.

int arr[max_columns]

(b) Two-dimensional Array

2D array can be defined as an array of arrays. The 2D array is organized as matrices which can be represented as the collection of rows and columns. However, 2D arrays are created to implement a relational database look alike data structure. It provides ease of holding bulk of data at once which can be passed to any number of functions wherever required.



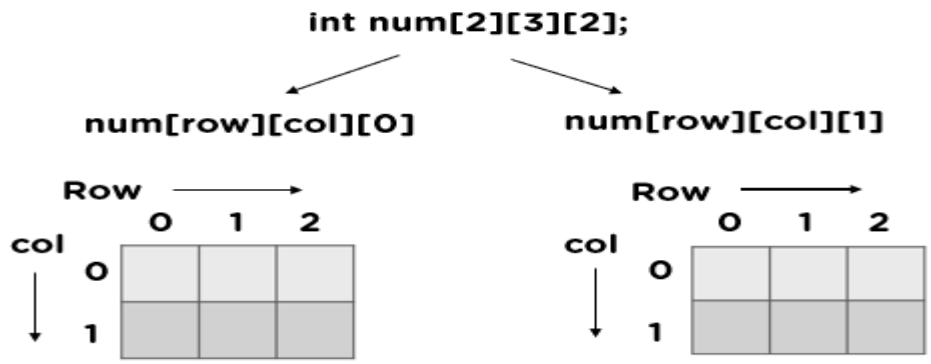
The syntax of declaring two-dimensional array is very much similar to that of a one-dimensional array, given as follows.

int arr[max_rows][max_columns]

(c) Multidimensional dimensional Array

When there is more than two dimensions of the array, it is called a multi-dimensional array. It can be 3D, or more dimensional. For example, the diagram below illustrates a three-dimensional array.

int a[size1][size2][size3]



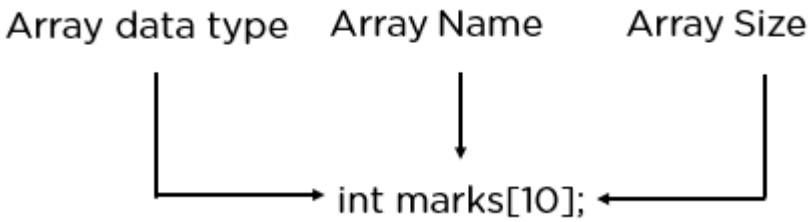
4.5.2 Dynamic vs Static Array

Static Arrays are arrays for which the size or length is determined when the array is created and/or allocated. For this reason, they may also be referred to as fixed-length arrays or fixed arrays. Size of static arrays are determined at compile-time (before run-time) and no need to delete static arrays because they are deleted automatically after going out of scope. Array values may be specified when the array is defined, or the array size may be defined without specifying array contents. Depending on the programming language, an uninitialized array may contain default values, or it may contain whatever values were left in memory from previous allocation.

Dynamic array is a random access, variable-size list data structure that allows elements to be added or removed. Dynamic Arrays are allocated on heap. Dynamic arrays overcome the limit of static arrays, which have a fixed capacity that needs to be specified at allocation. Dynamic arrays allow elements to be added and removed at runtime. Most current programming languages include built-in or standard library functions for creating and managing dynamic arrays.

4.6 Arrays Declaration

Arrays are typically defined with square brackets with the size of the arrays as its argument.



For One-dimensional array, the declaration is:

int arr[max_columns]

e.g.

Int a[10];

For Two-dimensional array, the declaration is:

int arr[max_rows][max_columns]

e.g.

int arr[5][5];

For Three-dimensional array, the declaration is:

int [size1][size2][size3];

e.g.

int a[5][5][5];

4.7 Manipulating the Content of Array (Arrays Operations)

The basic operations performed by an Arrays are:

(a) Traversal: print all the array elements one by one.
(b) Insertion: Adds an element at the given index.
(c) Deletion: Deletes an element at the given index.
(d) Search: Searches an element using the given index or by the value.
(e) Sorting: Arranging the elements of an Array in an order either in ascending or descending order.
(f) Update: Updates an element at the given index.
(g) Merging: Combining the contents of two Arrays

4.7.1 Traversal

Traversal is a process of processing or visiting each element in the array exactly once. Let A be an array stored in the computer's memory. If we want to display the contents of A, it has to be traversed i.e., by accessing and processing each element of A exactly once. The algorithm for Traversal is shown below:

Algorithm: (Traverse a Linear Array)

TRAVERSAL(LA, LB,UB, K)

[Here LA is a Linear array with lower boundary LB and upper boundary UB. This algorithm traverses LA applying an operation Process to each element of LA].

Step 1. [Initialize counter.] Set K ← LB.
Step 2. Repeat Steps 3 and 4 while K ≤ UB.
Step 3. [Visit element.] Apply PROCESS to LA[K].
Step 4. [Increase counter.] Set K ← K + 1.
[End of Step 2 loop.]
Step 5. Exit.

The alternate algorithm for traversing (using for loop) is:

Algorithm: (Traverse a Linear Array) This algorithm traverses a linear array LA with lower bound LB and upper bound UB.

Step 1. Repeat for K= LB to UB
Apply PROCESS to LA[K].
[End of loop].
Step 2. Exit.

4.7.2 Insertion

Insert operation insert one or more data elements into an array. Based on the requirement, a new element can be added at the beginning, end, or at any given index of the array. Let *A* be a collection of data elements in the memory of the computer. "Inserting" refers to the operation of adding another element to the collection A. Inserting an element at the "end" of a linear array can be easily done provided the memory space allocated for the array is large enough to accommodate the additional element. On the other hand, suppose we need to insert an element in the middle of the array. Then, on the average, half of the elements must be moved downward to new locations to accommodate the new location and keep the order of the other elements. The algorithm for Inserting.

Algorithm: (Inserting into a Linear Array)

INSERT (LA, N, K. ITEM)
[Here LA is a linear array with N elements and K is a positive integer such that K ≤ N. This algorithm inserts an element ITEM into the Kth position in LA].

Step I. [Initialize counter.] Set J← N.
Step 2. Repeat Steps 3 and 4 while I ≥ K.
Step 3. [Move J^{th} element downward.] Set LA [J + 1]←LA[J].
Step 4. [Decrease counter] Set J← J— 1.
[End of Step 2 loop.]
Step 5. [Insert clement] Set LA[K] ← ITEM.
Step 6. [Reset N.] Set N ← N+1
Step 7. Exit

4.7.3 Deletion

Deletion refers to removing an existing element from the array and re-organizing all elements of an array. Let *A* be a collection of data elements in the memory of the computer. "Deleting" refers to, the operation of removing

one of the elements from A. Deleting an element at the "end" of an array presents no difficulties, but deleting an element somewhere in the middle of the array would require that each subsequent element be moved one location upward in order to "fill up" the array.

Algorithm: Deleting from a Linear Array)

DELETE (LA, N, K, ITEM) Here LA is a linear array with N elements and K is a positive integer such that K ≤ N. This algorithm deletes the K^{th} clement from LA.

Step 1: Set ITEMLA ← (K)
Step 2: Repeat for I = K to N - 1:
[Move J + 1st clement upward.] Set LA(J) ← LA[J + I).
[End of loop,]
Step 3: [Reset the number N of elements in LA.) Set N: ← N - I.
Step 4: Exit.

4.7.4 Search

The process of finding a particular element of an array is called Searching. If the item is not present in the array, then the search is unsuccessful. A search can be performed on an array element based on its value or its index.

Algorithm

Consider *LA* is a linear array with N elements and K is a positive integer such that *K<=N*. the algorithm to find an element with a value of *ITEM* using sequential search.

SEARCH
Step 1. Start
Step 2. Set J=0
Step 3. Repeat steps 4 and 5 while J < N
Step 4. IF LA[J] is equal ITEM THEN GOTO STEP 6
Step 5. Set J = J +1
Step 6. PRINT J, ITEM
7. Stop

4.7.5 Update

Update operation refers to updating an existing element from the array at a given index.

Algorithm

Consider LA is a linear array with N elements and K is a positive integer such that K<=N. The algorithm updates an element available at the Kth position of LA.

UPDATE

Step 1. Start

Step 2. Set LA[K-1] = ITEM

Step 3. Stop

4.7.6 Merging

Merging two arrays means combining two separate arrays into one single array. For instance, if the first array consists of 3 elements and the second array consists of 5 elements then the resulting array consists of 8 elements. This resulting array is known as a merged array.

Algorithm:

MERGING

Step 1: Start

Step2: Create an array of size m+n named nums3[].

Step 3: Copy all elements of nums1[] to nums3[].

Step 4: Now traverse the elements of nums2[] and insert elements one by one to nums3[]

Step 5: Stop

4.8 Complexity of Array Operations

The Time complexity of various array operations are described in the following table.

Operation	**Average Case**	**Worst Case**
Access/Traversal	O (1)	O (1)
Search	O(n)	O(n)
Insertion	O(n)	O(n)
Deletion	O(n)	O(n)

In array, space complexity for worst case is **O(n)**.

4.9 Advantages and Disadvantages of Arrays

The advantages of Arrays are:

(a) Array provides the single name for the group of variables of the same type. Therefore, it is easy to remember the name of all the elements of an array.

(b) Traversing an array is a very simple process; we just need to increment the base address of the array in order to visit each element one by one.

(c) Any element in the array can be directly accessed by using the index.

The disadvantages of Arrays are:

(a) Array is homogenous. It means that the elements with similar data type can be stored in it.
(b) In array, there is static memory allocation that is size of an array cannot be altered.
(c) There will be wastage of memory if we store a smaller number of elements than the declared size.

4.10 Arrays Implementation in C++

In C++, an Array is a collection of elements of the same data type, stored in contiguous memory locations. Each element in the array can be accessed using an index. Arrays provide a convenient way to store and manipulate a group of related data items. For example, suppose a class has 27 students, and we need to store the grades of all of them. Instead of creating 27 separate variables, we can simply create an array:

double grade [27];

4.10.1 C++ Array Declaration

To declare an array in C++, the programmer specifies the type of the elements and the number of elements required by an array as follows

dataType arrayName[arraySize];

This is called a single-dimension array. The arraySize must be an integer constant greater than zero and type can be any valid C++ data type. For example, to declare a 10-element array called balance of type double, use this statement:

double balance [10];

Other examples are:

string cars[4]; // String array of size 4
int foo [5]; // Integer array of size 5

4.10.2 C++ Array Initialization

By default, regular arrays of local scope (for example, those declared within a function) are left uninitialized. This means that none of its elements are set to any particular value; their contents are undetermined at the point the array is declared. But the elements in an array can be explicitly initialized to specific values when it is declared, by enclosing those initial values in braces {}. For example:

```
// declare and initialize and array
int x[6] = {19, 10, 8, 17, 9, 15};
```

Other examples are:
string cars[4] = {"Volvo", "BMW", "Ford", "Mazda"};
int myNum[3] = {10, 20, 30};

Another method to initialize array during declaration:

```
// declare and initialize an array
int x[] = {19, 10, 8, 17, 9, 15};
```

for a 2-Dimensional Array,

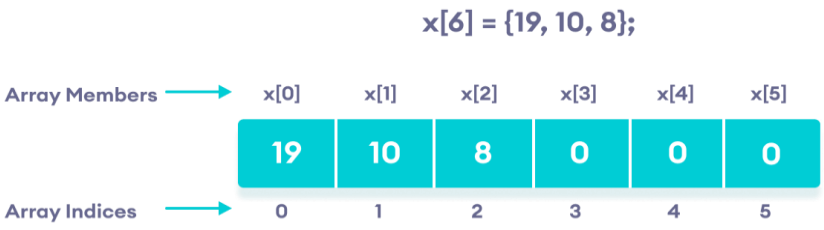
```
// Declaring and initializing a 2D array (matrix)
int matrix[3][3] = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9};
```

4.10.3 C++ Array with Empty Members

In C++, if an array has a size n, we can store upto n number of elements in the array. However, what will happen if we store less than n number of elements. For example,

```
// store only 3 elements in the array
int x[6] = {19, 10, 8};
```

Here, the array *x* has a size of 6. However, we have initialized it with only 3 elements. In such cases, the compiler assigns random values to the remaining places. Oftentimes, this random value is simply 0.



4.10.4 Access Elements in C++ Array

In C++, each element in an array is associated with a number. The number is known as an array index. We can access elements of an array by using those indices.

```
// syntax to access array elements
array[index];
e.g. balance [4] = 50.0;
```

```
// Modifying elements using the index
myArray[1] = 100; // Changes the 2nd element (20) to 100

// Accessing elements in a 2D array
int element = matrix[1][2]; // Accesses the element in the 2nd row and 3rd column (6)
```

Program Example: The program displays the elements of an array

```
#include <iostream>
using namespace std;
int main() {
int numbers[5] = {7, 5, 6, 12, 35};
cout << "The numbers are: ";
// using traditional for loop
for (int i = 0; i < 5; ++i) {
cout << numbers[i] << "  ";
}
return 0;
}
```

4.11 Arrays Implementation in Java

In Java, an array is a data structure that can store a fixed-size sequence of elements of the same data type. An array is an object in Java, which means it can be assigned to a variable, passed as a parameter to a method, and returned as a value from a method. Arrays in Java are zero-indexed, which means that the first element in an array has an index of 0, the second element has an index of 1, and so on. The length of an array is fixed when it is created and cannot be changed later. For example, if we want to store the names of 100 people then we can create an array of the string type that can store 100 names.

```
String [] array = new String [100];
```

Java arrays can store elements of any data type, including primitive types such as int, double, and boolean, as well as object types such as String and Integer. Arrays can also be multi-dimensional, meaning that they can have

multiple rows and columns. Arrays in Java are commonly used to store collections of data, such as a list of numbers, a set of strings, or a series of objects. By using arrays, we can access and manipulate collections of data more efficiently than using individual variables.

4.11.1 Declaration of an array in Java

To use an array in a program, the variable that reference the array must be declare, and the type of array the variable can reference must also specify. Here is the syntax for declaring an array variable:

dataType[] arrayName;
or
dataType arrayName[];

The style *dataType[] arrayName* is preferred. The style *dataType arrayName []* comes from the C/C++ language and was adopted in Java to accommodate C/C++ programmers.

dataType can be primitive data types like int, char, double, byte, etc. or Java objects and arrayName must be an identifier. The following code snippets are examples of this syntax:

```
double[] myList;   // preferred way.
double myList[];   // works but not preferred way.
String[] cars;
```

4.11.2 Creating Arrays

An array can be created by using the new operator with the following syntax: The Syntax for creating the Array is:

arrayRefVar = new dataType[arraySize];

The above statement does two things; It creates an array using new dataType[arraySize] and it assigns the reference of the newly created array to the variable arrayRefVar.

Declaring an array variable, creating an array, and assigning the reference of the array to the variable can be combined in one statement, as shown below:

dataType[] arrayRefVar = new dataType[arraySize];

Alternatively, you can create arrays as follows:

dataType[] arrayRefVar = {value0, value1, ..., valuek};

the statement below declares an array variable and myList, creates an array of 10 elements of double type and assigns its reference to myList:

double[] myList = new double[10];

4.11.3 Initializing an Array

In Java, we can initialize arrays during declaration. For example,

```
//declare and initialize and array
int[] age = {12, 4, 5, 2, 5};
```

Here, we have created an array named age and initialized it with the values inside the curly brackets. Other examples are:

```
String[] cars = {"Volvo", "BMW", "Ford", "Mazda"};
float[] myNum = {10, 20, 30, 40};
```

4.11.4 Array Length

To find out how many elements an array has, use the *length* property:

```
String[] cars = {"Volvo", "BMW", "Ford", "Mazda"};
System.out.println(cars.length);
// Outputs 4
```

4.11.5 Accessing and Processing Elements of an Array in Java

We can access the element of an array using the index number. Here is the syntax for accessing elements of an array:

```
// access array elements
array[index]
```

For example, the program below accesses the elements of an array

```
class Main {
public static void main(String[] args) {
 // create an array
        int[] age = {12, 4, 5, 2, 5};
        // access each array elements
        System.out.println("Accessing Elements of Array:");
        System.out.println("First Element: " + age[0]);
        System.out.println("Second Element: " + age[1]);
```

```
            System.out.println("Third Element: " + age[2]);
            System.out.println("Fourth Element: " + age[3]);
            System.out.println("Fifth Element: " + age[4]);
        }
    }
```

When processing array elements, we often use either for loop or foreach loop because all of the elements in an array are of the same type and the size of the array is known. The examples below illustrate on how loop with arrays.

```
    class Main {
    public static void main(String[] args) {
    // create an array
    int[] age = {12, 4, 5};

    // loop through the array
    // using for loop
    System.out.println("Using for Loop:");
    for(int i = 0; i < age.length; i++) {
    System.out.println(age[i]);
    }
    }
}
```

Here is a complete example showing how to create, initialize, and process arrays

```
    public class TestArray {
        public static void main(String[] args) {
            double[] myList = {1.9, 2.9, 3.4, 3.5};
            // Print all the array elements
            for (int i = 0; i < myList.length; i++) {
                System.out.println(myList[i] + " ");
            }
                 // Summing all elements
                double total = 0;
            for (int i = 0; i < myList.length; i++) {
                total += myList[i];
            }
            System.out.println("Total is " + total);
                 // Finding the largest element
            double max = myList[0];
            for (int i = 1; i < myList.length; i++) {
            if (myList[i] > max) max = myList[i];
```

```
            }
            System.out.println("Max is " + max);
        }
    }
```

4.11.6 The foreach Loops

JDK 1.5 introduced a new for loop known as foreach loop or enhanced for loop, which enables you to traverse the complete array sequentially without using an index variable. The following codes displays all the elements in the array myList:

```
public class TestArray {
    public static void main(String[] args) {
        double[] myList = {1.9, 2.9, 3.4, 3.5};
 // Print all the array elements
        for (double element: myList) {
        System.out.println(element);
        }
    }
}
```

4.11.7 Passing Arrays to Methods

Just as you can pass primitive type values to methods, you can also pass arrays to methods. For example, the following method displays the elements in an int array: The example illustrate how to pass arrays to methods.

```
public static void printArray(int[] array) {
    for (int i = 0; i < array.length; i++) {
    System.out.print(array[i] + " ");
    }
}
```

You can invoke it by passing an array. For example, the following statement invokes the printArray method to display 3, 1, 2, 6, 4, and 2.

```
printArray(new int[]{3, 1, 2, 6, 4, 2});
```

4.11.8 Returning an Array from a Method

A method may also return an array. For example, the following method returns an array that is the reversal of another array:

```
public static int[] reverse(int[] list) {
    Int[] result = new int[list.length];
```

```
        for (int i = 0, j = result.length - 1; i < list.length; i++, j--) {
            result[j] = list[i];
        }
        return result;
    }
```

Program Example: Compute Sum and Average of Array Elements

```
        class Main {
            public static void main(String[] args) {
            int[] numbers = {2, -9, 0, 5, 12, -25, 22, 9, 8, 12};
            int sum = 0;
            double average;
            // access all elements using for each loop
            // add each element in sum
            for (int number: numbers) {
                sum += number;
            }
            // get the total number of elements
            int arrayLength = numbers.length;
            // calculate the average
            // convert the average from int to double
            average =  ((double)sum / (double)arrayLength);
            System.out.println("Sum = " + sum);
            System.out.println("Average = " + average);
        }
    }
```

4.12 Hands-on on Array Operation Algorithms (using C++ or Java)

- Implement the array operations (Traversal, Inserting, deleting, searching, merging and updating) using either C++ or Java

4.13 Assignment on Array Implementation

(a) Write an Algorithm to compute the sum of the two given arrays of integers. Implement the algorithm using either C++ or Java.

(b) Write an algorithm to create an array by swapping the first and last elements of a given array of integers. Implement the algorithm using either C++ or Java.

(c) Write an Algorithm to count the even number of elements in a given array of integers. Implement the algorithm using either C++ or Java.

(d) Write an Algorithm to check whether a given array of integers contains 5's and 7's. Implement the algorithm using either C++ or Java.

(e) Write an Algorithm to count the numbers of odd and even in an array. Implement the algorithm using either C++ or Java.

(f) The Fibonacci series is a special series in Mathematics. Write an algorithm to compute the first 20 terms of the Fibonacci series given by: 0, 4, 4, 8, 12, …